

Spring Boot Essentials

3-Day Workshop: Building Scalable APIs

Training Roadmap

Day 1: Fundamentals

- Spring Boot Setup & Auto-Config
- Inversion of Control (IoC)
- Dependency Injection (@Autowired)
- Bean Scopes (Singleton vs Prototype)

Day 2: REST API

- REST Principles & Methods
- @RestController & @RequestMapping
- GET: Retrieving Data (@PathVariable)
- POST: Creating Data (@RequestBody)

Day 3: Config & Testing

- | | | | |
|--------------------------|--------------------------|----------------------|--------------------|
| • Properties &
@Value | • Profiles
(Dev/Prod) | • Postman
Testing | • Final
Project |
|--------------------------|--------------------------|----------------------|--------------------|

Day 1

Fundamentals & Dependency Injection

Module 8: Spring Boot Setup

Why Spring Boot?

Opinionated configuration to get you started quickly.

- **Starters:** Bundled dependencies (e.g., spring-boot-starter-web).
- **Embedded Server:** Tomcat included.
- **Auto-Configuration:** Configures beans based on classpath.



Module 8: Spring Boot Setup

Simplified Application Setup

Auto-configuration detects dependencies and applies defaults to reduce boilerplate and speed up startup.

Spring Boot Starters

Starters bundle curated dependencies, preventing version conflicts and easing REST API development.

@SpringBootApplication Annotation

Combines key annotations to simplify bootstrapping and component scanning in Spring Boot projects.

Customization and Troubleshooting



Module 8: Spring Boot Setup

The `@SpringBootApplication` annotation is actually a **convenience annotation** that wraps three other essential annotations into one. When you place it on your main class, you are effectively doing three things at once:

1. `@Configuration`

This tells Spring that the class can be used as a source of **Bean definitions**. It allows you to use the `@Bean` annotation inside the class to manually register objects into the Spring Context.

2. `@EnableAutoConfiguration`

This is where the "magic" happens. It tells Spring Boot to start adding beans based on the libraries it finds on your **classpath**.

- **Example:** If Spring finds `spring-boot-starter-web` in your project, this annotation automatically configures an embedded Tomcat server and sets up Spring MVC for you.

3. `@ComponentScan`

This tells Spring to go look for other classes marked with `@Component`, `@Service`, `@Repository`, or `@RestController`.

- **Important Note:** By default, it only scans the package where your main class is located and all of its **sub-packages**. This is why your main class should always be at the "root" of your project structure.

Exercise 1 : Project Initialization

Hands-on Activity

Objective: Create your first Spring Boot application.

- Go to **start.spring.io**.
- Select Maven/Gradle and Java.
- Add Dependency: **Spring Web**.
- Generate and unzip the project.
- Open in your IDE and run the main method.
- Verify Tomcat started on port 8080 in the console.

Module 8: Dependency Injection



Module 8: Dependency Injection

Inversion of Control (IoC)

The Spring Container manages object lifecycle.
You don't use new; you let Spring create objects (Beans) for you.

```
@Component  
public class BookService { ... }
```

@Autowired Injection

Spring injects the required dependency automatically.

```
@RestController  
public class BookController {  
    // Constructor Injection (Best Practice)  
    private final BookService service;  
  
    public BookController(BookService s) {  
        this.service = s;  
    }  
}
```

Module 8: Dependency Injection

Core Concept of DI

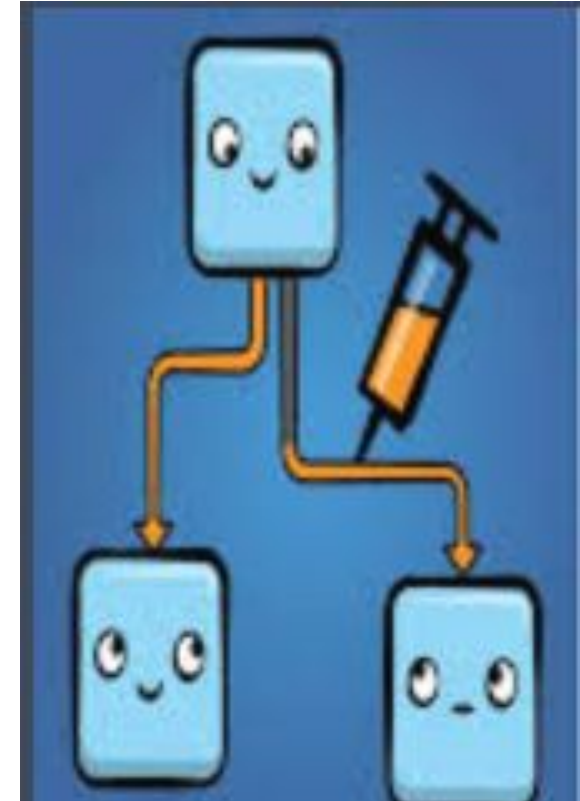
Dependency Injection allows supplying required components externally to promote loose coupling and modular design.

@Autowired Usage

The @Autowired annotation instructs Spring Boot to automatically inject needed beans into constructors, fields, or setters.

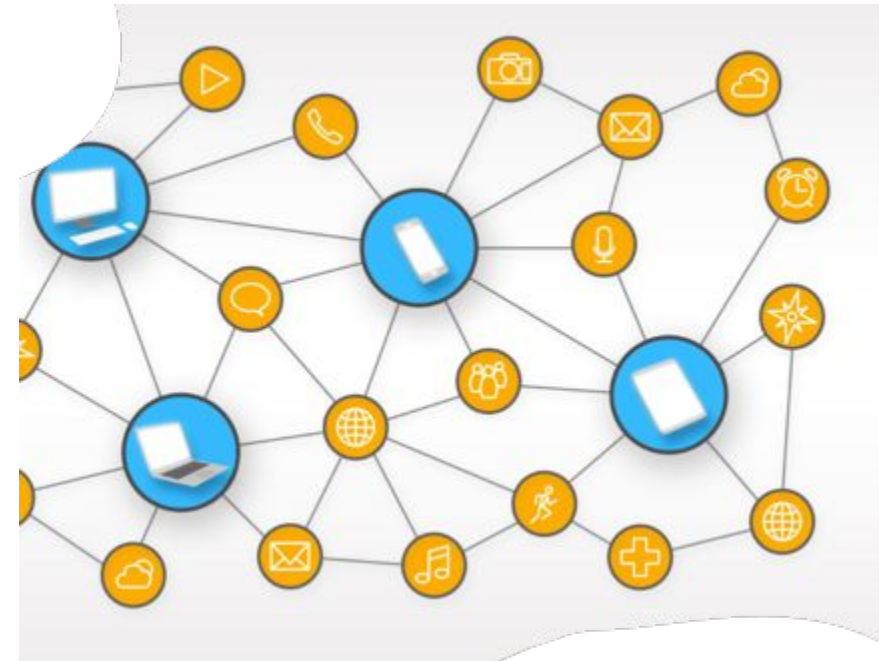
Benefits and Best Practices

Constructor injection is preferred for better immutability, testability, and clear dependency declaration in Spring Boot.



Module 8: Dependency Injection

1. Constructor Injection
2. Field Injection
3. Setter Injection



Module 8: Dependency Injection

- ✓ 1. Constructor Injection (★ Recommended – Best Practice)
This is the preferred way in modern Spring (including Spring Boot).

Example Service

@Service

```
public class EmailService {  
  
    public void sendEmail() {  
        System.out.println("Sending email...");  
    }  
}
```

Why Use This?

- ✓ Makes dependencies explicit
- ✓ Easier testing (mocking)
- ✓ Works well with final fields
- ✓ Prevents null dependencies

👉 Best for production code

Component (Using Constructor Injection)

@Component

```
public class NotificationService {  
  
    private final EmailService emailService;  
  
    // Constructor Injection  
    public NotificationService(EmailService emailService) {  
        this.emailService = emailService;  
    }  
  
    public void notifyUser() {  
        emailService.sendEmail();  
    }  
}
```

Module 8: Dependency Injection

✓ 2. Field Injection (⚠ Not Recommended)

Spring injects dependencies directly into fields.

@Service

```
public class NotificationService {
```

@Autowired

```
    private EmailService emailService;
```

```
    public void notifyUser() {  
        emailService.sendEmail();  
    }  
}
```

Problems

- ✗ Hard to test
- ✗ Hidden dependencies
- ✗ Cannot use final
- ✗ Poor design

👉 Avoid in serious projects

✓ 3. Setter Injection

Dependency is injected through a setter method.

@Component

```
public class NotificationService {
```

```
    private EmailService emailService;
```

@Autowired

```
    public void setEmailService(EmailService emailService) {  
        this.emailService = emailService;  
    }  
}
```

```
    public void notifyUser() {  
        emailService.sendEmail();  
    }  
}
```

When to Use

- ✓ When dependency is optional
- ✓ When you want to change it later

Exercise 2: Injecting Dependencies

🔗 Hands-on Activity (Constructor based)

Objective: Wire components together using DI.

- Create a class `LoggerService` annotated with `@Component` or `@Service`.
- Add a method `log(String msg)` that prints to console.
- Create a `BookService` class and inject `LoggerService` using Constructor Injection.
- Call the `log` method inside `BookService`.
- Verify the output on the console when the app runs.

Exercise 3: Injecting Dependencies

🔗 Hands-on Activity (Setter Injection)

Objective: Wire components together using DI.

- Create a class `LoggerService` annotated with `@Component` or `@Service`.
- Add a method `log(String msg)` that prints to console.
- Create a `BookService` class and inject `LoggerService` using Constructor Injection.
- Call the `log` method inside `BookService`.
- Verify the output on the console when the app runs.

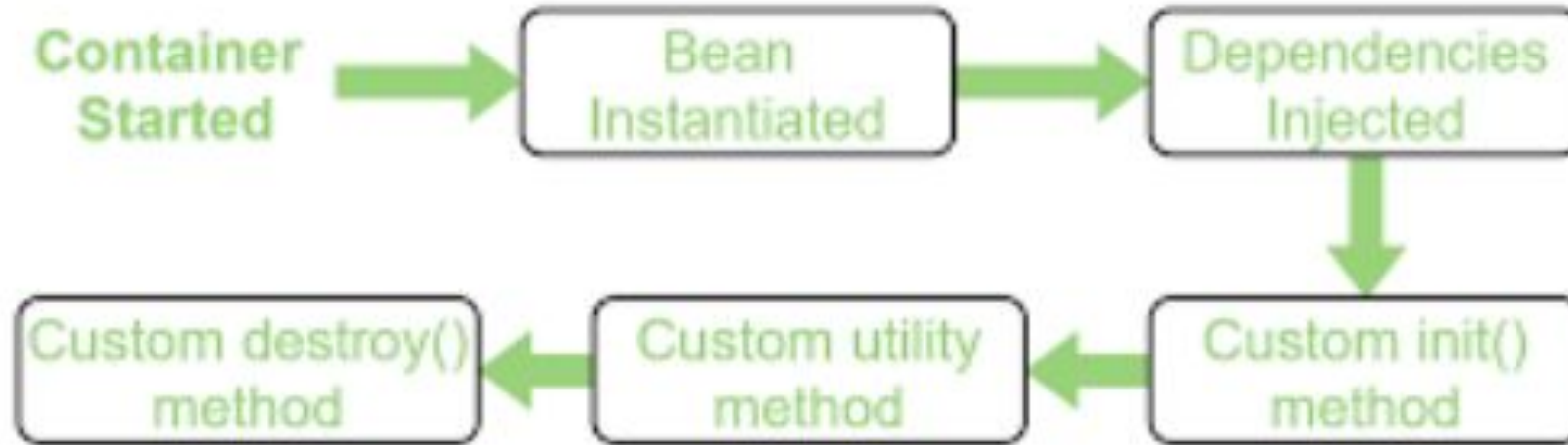
Exercise 4: Injecting Dependencies

🔗 Hands-on Activity (Field Injection)

Objective: Wire components together using DI.

- Create a class `LoggerService` annotated with `@Component` or `@Service`.
- Add a method `log(String msg)` that prints to console.
- Create a `BookService` class and inject `LoggerService` using Constructor Injection.
- Call the `log` method inside `BookService`.
- Verify the output on the console when the app runs.

Module 8: Bean Life Cycle



Module 8: Bean Scopes

Singleton Scope Overview

Singleton scope creates one shared bean instance throughout the application lifecycle, ideal for stateless services and shared resources.

Prototype Scope Usage

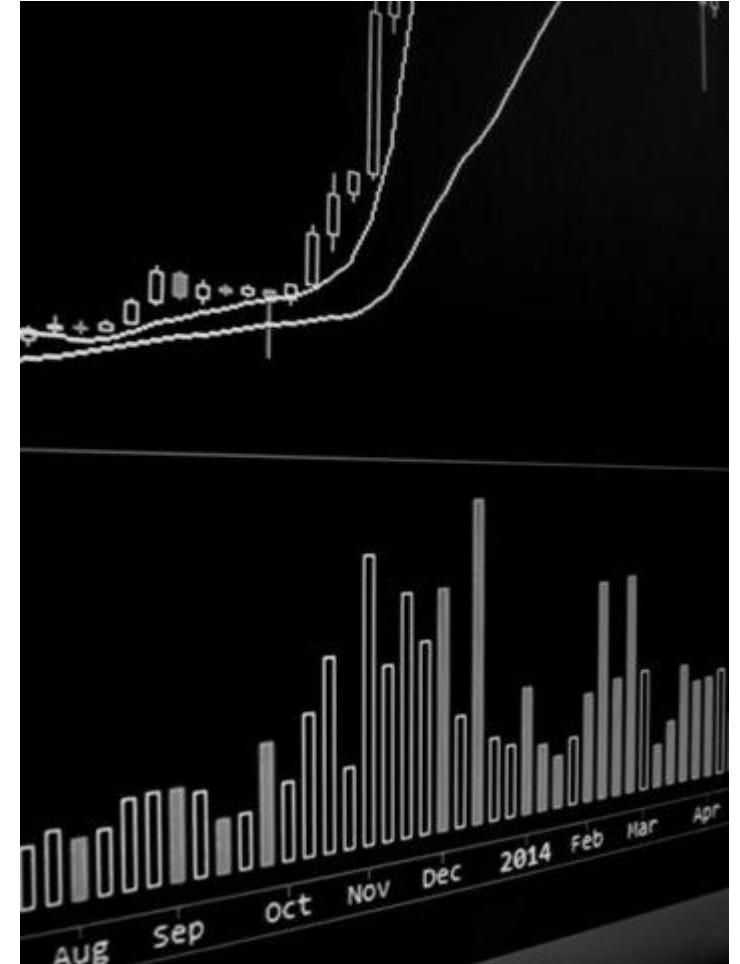
Prototype scope creates a new bean instance each time it is requested, useful for stateful or temporary objects.

Web Scopes: Request and Session

Request and session scopes associate beans with individual HTTP requests or user sessions for finer lifecycle control.

Scope Configuration and Proxies

Using annotations and proxies ensures proper lifecycle handling and avoids stale or shared state issues



Module 8: Bean Scopes

Singleton Scope Example (Default)

 **Bean: SingletonService**

```
import org.springframework.stereotype.Component;
```

@Component

```
public class SingletonService {  
  
    public SingletonService() {  
        System.out.println("SingletonService Created");  
    }  
}
```

No scope annotation = singleton by default

Prototype Scope Example

 **Bean: PrototypeService**

```
import org.springframework.context.annotation.Scope;  
import org.springframework.stereotype.Component;
```

@Component

@Scope("prototype")

```
public class PrototypeService {  
  
    public PrototypeService() {  
        System.out.println("PrototypeService Created");  
    }  
}
```

@Scope("prototype") → New instance every time

Module 8: Bean Scopes

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
```

@SpringBootApplication

```
public class DemoApplication {

    public static void main(String[] args) {

        ApplicationContext context =
            SpringApplication.run(DemoApplication.class, args);

        // Get Singleton Beans
        SingletonService s1 = context.getBean(SingletonService.class);
        SingletonService s2 = context.getBean(SingletonService.class);

        // Get Prototype Beans
        PrototypeService p1 = context.getBean(PrototypeService.class);
        PrototypeService p2 = context.getBean(PrototypeService.class);

        System.out.println("Singleton Same? " + (s1 == s2));
        System.out.println("Prototype Same? " + (p1 == p2));
    }
}
```

Exercise 6: Singleton and Prototype

Hands-on Activity

Objective: Demonstrate the difference between Singleton and Prototype

- Create a class SingletonService and a PrototypeService annotated with @Component or @Service.
- Add a method print() that prints to console.
- Create a DemoService and create 2 instance of each service class
- Call the print method in DemoService
- Verify the output in the console when the app runs.